

THE REAL-WORLD STORY

Imagine standing in line at a movie theater ticket counter. The first person to arrive is the first person served (First-In, First-Out: FIFO).

THE QUEUE & DEQUE AHA MOMENT

A Double-Ended Queue (Deque) allows **O(1) insertions and deletions at BOTH ends** ('head' and 'tail'), making it the ultimate foundation for BFS tree/graph traversals and monotonic sliding window maximums!

WHY ARRAYLIST FAILS FOR QUEUES

- ArrayList Remove First**
`list.remove(0)`: Requires shifting all remaining N elements → Slow O(N) time!
- ArrayDeque pollFirst()**: Uses a circular array buffer → Strict **O(1) time!**
- Java Recommendation:** ALWAYS use `ArrayDeque` instead of legacy `Stack` or `LinkedList`!

VISUALIZING DOUBLE-ENDED QUEUE

Deque Operations (Both Ends O(1)):

```
addFirst(x)  →  
← addLast(x)  
pollFirst() ← [ Head | ... |  
Tail ] → pollLast()  
peekFirst()  ^^^^^  
^^^^^ peekLast()
```

Double-ended power: Acts as both Stack & Queue!

PREREQUISITES & COMPLEXITY REASONING

Data Structure / Pattern	Operation	Time Complexity	Auxiliary Space
<code>ArrayDeque</code>	<code>offer()</code> , <code>poll()</code> , <code>peek()</code>	O(1) amortized	O(N) circular buffer
BFS Level Order Traversal	<code>queue.offer()</code> & <code>poll()</code>	O(V + E)	O(V) queue capacity
Monotonic Deque (Max Window)	<code>pollLast()</code> , <code>offerLast()</code>	O(N) single pass	O(K) deque size
Circular Queue Implementation	<code>enqueue()</code> , <code>dequeue()</code>	O(1) strictly	O(K) fixed array

CLUES THAT SCREAM QUEUE / DEQUE

- "Level order traversal / Shortest path in unweighted graph" → Standard BFS Queue (`ArrayDeque`)
- "Maximum/minimum in every sliding window of size K" → Monotonic Decreasing Deque
- "First non-repeating character in a stream" → Queue + Character Frequency Map

PATTERN
1

BFS LEVEL ORDER TRAVERSAL (Queue Snapshot Loop)

e.g. Binary Tree Level Order Traversal (LC 102)

LEVEL SNAPSHOT STRATEGY

At the start of each level loop, capture `int size = queue.size()`. Process EXACTLY `size` nodes for the current level before advancing to the next level!

TEMPLATE — LC 102

```
public List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> res = new ArrayList<>();
    if (root == null) return res;
    Queue<TreeNode> queue = new ArrayDeque<>();
    queue.offer(root);
    while (!queue.isEmpty()) {
        int levelSize = queue.size(); // Snapshot size for current level!
        List<Integer> currentLevel = new ArrayList<>();
        for (int i = 0; i < levelSize; i++) {
            TreeNode node = queue.poll();
            currentLevel.add(node.val);
            if (node.left != null) queue.offer(node.left);
            if (node.right != null) queue.offer(node.right);
        }
        res.add(currentLevel);
    }
    return res;
}
```

PATTERN 2 MONOTONIC DEQUE (Sliding Window Maximum) e.g. Sliding Window Maximum (LC 239)

MONOTONIC DECREASING ORDER

Store array indices in deque. Maintain decreasing order of values. Pop smaller tail elements before adding `i`. Front of deque is ALWAYS the max element of current window!

TEMPLATE — LC 239

```
public int[] maxSlidingWindow(int[] nums, int k) {
    int n = nums.length;
    int[] res = new int[n - k + 1];
    Deque<Integer> dq = new ArrayDeque<>();
    for (int i = 0; i < n; i++) {
        // 1. Remove elements outside current window range:
        if (!dq.isEmpty() && dq.peekFirst() < i - k + 1) dq.pollFirst();
        // 2. Remove smaller elements from tail:
        while (!dq.isEmpty() && nums[dq.peekLast()] < nums[i]) dq.pollLast();
        dq.offerLast(i);
        // 3. Record max when window size ≥ k:
        if (i ≥ k - 1) res[i - k + 1] = nums[dq.peekFirst()];
    }
    return res;
}
```

PATTERN 3

STREAM QUEUE (First Unique Character in String) e.g. First Unique Character in a String (LC 387)

FREQUENCY ARRAY + QUEUE

Count frequencies in `int[26]`. Offer characters into queue. Poll from front while `count[queue.peek()] > 1`!

TEMPLATE — LC 387

```
public int firstUniqChar(String s) {
    int[] count = new int[26];
    for (int i = 0; i < s.length(); i++) count[s.charAt(i) - 'a']++;
    for (int i = 0; i < s.length(); i++) {
        if (count[s.charAt(i) - 'a'] == 1) return i;
    }
    return -1;
}
```

PATTERN
4

DESIGN CIRCULAR QUEUE (Fixed Array + Modulo
Head/Tail)

e.g. Design Circular Queue (LC
622)

MODULO ARRAY BUFFER

Use fixed array `a[capacity]`. Increment `tail = (tail + 1) % capacity` on `enqueue()`, and `head = (head + 1) % capacity` on `dequeue()`.

TEMPLATE — LC 622

```
class MyCircularQueue {
    private int[] a;
    private int head = 0, tail = 0, count = 0, cap;
    public MyCircularQueue(int k) { cap = k; a = new int[k]; }
    public boolean enqueue(int val) {
        if (isFull()) return false;
        a[tail] = val; tail = (tail + 1) % cap; count++; return true;
    }
    public boolean dequeue() {
        if (isEmpty()) return false;
        head = (head + 1) % cap; count--; return true;
    }
    public int Front() { return isEmpty() ? -1 : a[head]; }
    public int Rear() { return isEmpty() ? -1 : a[(tail - 1 + cap) % cap]; }
    public boolean isEmpty() { return count == 0; }
    public boolean isFull() { return count == cap; }
}
```

COMPLETE QUEUE & DEQUE PROBLEM LADDER SUMMARY

LC #	Problem	Pattern	Time	Space	Diff
387	First Unique Character in String	Character Frequency Scan	O(N)	O(1)	E
102	Binary Tree Level Order Traversal	BFS Queue Snapshot Loop	O(N)	O(W)	M
622	Design Circular Queue	Fixed Array + Modulo Pointers	O(1)	O(K)	M
225	Implement Stack using Queues	Single Queue Rotation Loop	O(N) push	O(N)	E
232	Implement Queue using Stacks	2-Stack Inversion	O(1) amortized	O(N)	E
103	Binary Tree Zigzag Level Order	BFS + Deque Alternating Insert	O(N)	O(W)	M
239	Sliding Window Maximum	Monotonic Decreasing Deque	O(N)	O(K)	H

LC 387 · First Unique Character

EASY

Pattern: Freq Array Scan
Key Insight: `int[26]` frequency count.

```
public int firstUniqChar(String s) {
    int[] c = new int[26];
    for (int i = 0; i < s.length(); i++) c[s.charAt(i) - 'a']++;
    for (int i = 0; i < s.length(); i++) if (c[s.charAt(i) - 'a'] == 1) return i;
    return -1;
}
```

LC 102 · Binary Tree Level Order Traversal

MEDIUM

Pattern: BFS Queue Snapshot
Key Insight: Loop `int sz = q.size()`.

```
public List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> res = new ArrayList<>(); if (root == null) return res;
    Queue<TreeNode> q = new ArrayDeque<>(); q.offer(root);
    while (!q.isEmpty()) {
        int sz = q.size(); List<Integer> cur = new ArrayList<>();
        for (int i = 0; i < sz; i++) {
            TreeNode n = q.poll(); cur.add(n.val);
            if (n.left != null) q.offer(n.left); if (n.right != null) q.offer(n.right);
        }
        res.add(cur);
    }
    return res;
}
```

LC 622 · Design Circular Queue

MEDIUM

Pattern: Modulo Pointer Array
Key Insight: `tail = (tail + 1) % cap`.

```
class MyCircularQueue {
    int[] a; int head = 0, tail = 0, count = 0, cap;
    public MyCircularQueue(int k) { cap = k; a = new int[k]; }
    public boolean enqueue(int val) {
        if (isFull()) return false;
        a[tail] = val; tail = (tail + 1) % cap; count++; return true;
    }
    public boolean dequeue() {
        if (isEmpty()) return false;
        head = (head + 1) % cap; count--; return true;
    }
    public boolean isEmpty() { return count == 0; }
    public boolean isFull() { return count == cap; }
}
```

LC 225 · Implement Stack using Queues

EASY

Pattern: Single Queue Rotation
Key Insight: Rotate `sz - 1` items to back.

```

class MyStack {
    Queue<Integer> q = new LinkedL
ist<>();
    public void push(int x) {
        q.offer(x);
        for (int i = 0; i < q.size()
- 1; i++) q.offer(q.poll());
    }
    public int pop() { return q.po
ll(); }
    public int top() { return q.pe
ek(); }
    public boolean empty() { retur
n q.isEmpty(); }
}

```

LC 232 · Implement Queue using Stacks

EASY

Pattern: 2-Stack Inversion
Key Insight: Transfer `in` stack to `out` stack.

```

class MyQueue {
    Stack<Integer> in = new Stack<
>(), out = new Stack<>();
    public void push(int x) { in.p
ush(x); }
    public int pop() { peek(); ret
urn out.pop(); }
    public int peek() {
        if (out.isEmpty()) while (!i
n.isEmpty()) out.push(in.pop());
        return out.peek();
    }
    public boolean empty() { retur
n in.isEmpty() && out.isEmpty();
}
}

```

LC 239 · Sliding Window Maximum

HARD

Pattern: Monotonic Deque
Key Insight: Deque front stores window max index.

```

public int[] maxSlidingWindow(int
[] nums, int k) {
    int n = nums.length; int[] res =
new int[n - k + 1];
    Deque<Integer> dq = new ArrayDeq
ue<>();
    for (int i = 0; i < n; i++) {
        if (!dq.isEmpty() && dq.peekFi
rst() < i - k + 1) dq.pollFirst();
        while (!dq.isEmpty() && nums[d
q.peekLast()] < nums[i]) dq.pollLa
st();
        dq.offerLast(i);
        if (i ≥ k - 1) res[i - k + 1]
= nums[dq.peekFirst()];
    }
    return res;
}

```

LC 103 · Binary Tree Zigzag Level Order

MEDIUM

Pattern: BFS + Deque Insertion
Key Insight: Alternate level insertion direction.

```

public List<List<Integer>> zigzagL
evelOrder(TreeNode root) {
    List<List<Integer>> res = new Ar
rayList<>(); if (root == null) ret
urn res;
    Queue<TreeNode> q = new ArrayDeq
ue<>(); q.offer(root); boolean lTo
R = true;
    while (!q.isEmpty()) {
        int sz = q.size(); LinkedList<
Integer> level = new LinkedList<>
();
        for (int i = 0; i < sz; i++) {
            TreeNode n = q.poll();
            if (lToR) level.addLast(n.va
l); else level.addFirst(n.val);
            if (n.left != null) q.offer
(n.left); if (n.right != null) q.o
ffer(n.right);
        }
        res.add(level); lToR = !lToR;
    }
    return res;
}

```

🔍 DRY RUN — Circular Queue Enqueue & Dequeue

Step	Operation	head	tail	count	Array State (cap = 3)
1	<code>enQueue(10)</code>	0	1	1	<code>[10, _, _]</code>
2	<code>enQueue(20)</code>	0	2	2	<code>[10, 20, _]</code>
3	<code>deQueue()</code>	1	2	1	<code>[_ , 20, _]</code>
4	<code>enQueue(30)</code>	1	0 (mod 3)	2	<code>[30, 20, _]</code> ✅ Modulo Wrap!

📐 MATHEMATICAL PROOF — O(1) AMORTIZED TIME OF 2-STACK QUEUE

Claim: 2-Stack Queue `pop()` and `peek()` run in O(1) amortized time.

Proof: Each element is pushed to `in` stack once, popped from `in` stack once, pushed to `out` stack once, and popped from `out` stack once. Total 4 operations per element over N calls $\implies 4N / N = 4 = \mathbf{O(1)}$ Amortized Time!

✅ MASTERY CHECKLIST — CAN YOU DO THIS?

☐ Write BFS Level Order Traversal with queue snapshot

☐ Implement Circular Queue using array and modulo arithmetic

☐ Code Implement Stack using single Queue rotation

☐ Code Implement Queue using 2 Stacks in O(1) amortized

☐ Write Monotonic Deque for Sliding Window Maximum in O(N)

☐ Code Zigzag Level Order Traversal with Deque insertion

☐ Explain why `ArrayDeque` outperforms `LinkedList` in Java

☐ Prove why 2-stack queue runs in O(1) amortized time

📦 GOLDEN RULES — NEVER FORGET

Rule 1 — Always Snapshot Queue Size for BFS Levels
In BFS level order traversals, ALWAYS snapshot `int size = queue.size()` at the top of the outer loop before starting the inner loop. Do NOT evaluate `queue.size()` dynamically inside the inner loop!

Rule 2 — Deque Stores Indices for Range Pruning
In Monotonic Deque problems, ALWAYS store array indices instead of values so you can easily prune elements that fall outside the active window (`dq.peekFirst() < i - k + 1`).